```
#########################################################################################################
# Bangkok Flood Map – 26/27 September 2026
# Districts filled by flood severity (from satellite) + Gauge stations + High-resolution inundation
# All English labels
# Author: Atiq Kainan Ahmed. Email: atiqka@yahoo.com
#########################################################################################################

from pathlib import Path
import zipfile
import requests
import numpy as np
import pandas as pd
import geopandas as gpd
from shapely.geometry import Point, mapping
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
from matplotlib.lines import Line2D
import rasterio
from rasterio.mask import mask
from rasterio.plot import show
from concurrent.futures import ThreadPoolExecutor, as_completed

# ---------------------------------------------------------------------
# Settings
# ---------------------------------------------------------------------
ZIP_PATH     = Path("EOSRS_Flood_Proxy_Map_Sep2026.zip")
EXTRACT_DIR  = Path("eosrs_flood_sep2026")
DISTRICTS    = Path("thailand_districts.geojson")
OUTPUT_PNG   = "Bangkok_Districts_Filled_by_Flood_Severity.png"
TARGET_DATE  = "2026-09-26"
MAX_WORKERS  = 10

# ---------------------------------------------------------------------
# 1. Districts
# ---------------------------------------------------------------------
DISTRICTS_URL = "https://raw.githubusercontent.com/chingchai/OpenGISData-Thailand/master/districts.geojson"

if not DISTRICTS.exists():
    print("Downloading districts...")
    r = requests.get(DISTRICTS_URL, timeout=60)
    r.raise_for_status()
    DISTRICTS.write_bytes(r.content)

gdf_dist = gpd.read_file(DISTRICTS)
bkk = gdf_dist[gdf_dist["pro_code"] == "10"].copy().sort_values("amp_en")
print(f"Bangkok districts: {len(bkk)}")

# ---------------------------------------------------------------------
# 2. High-resolution flood raster
# ---------------------------------------------------------------------
HDX_URL = (
    "https://data.humdata.org/dataset/e6084162-f3c6-478e-9096-33522e15f283"
    "/resource/80dfdb12-0855-4719-afca-e2b01b564b3a"
    "/download/eosrs_2026_016_tha_fl_20260929_fpm_s1_bangkok_v0.8.zip"
```

```
)

def download_flood_map():
    print("Downloading high-resolution flood map...")
    headers = {"User-Agent": "Mozilla/5.0"}
    with requests.get(HDX_URL, headers=headers, stream=True, timeout=180, allow_redirects=True) as r:
        if r.status_code != 200:
            raise RuntimeError(f"Download failed: {r.status_code}")
        ct = (r.headers.get("Content-Type") or "").lower()
        if "html" in ct or "text" in ct:
            raise RuntimeError("Server returned HTML. Download ZIP manually from HDX.")
        with open(ZIP_PATH, "wb") as f:
            for chunk in r.iter_content(8192):
                if chunk:
                    f.write(chunk)
    print("Download complete.")

if (not ZIP_PATH.exists() or ZIP_PATH.stat().st_size < 100_000
        or not zipfile.is_zipfile(ZIP_PATH)):
    download_flood_map()
else:
    print("Flood map ZIP already exists.")

EXTRACT_DIR.mkdir(exist_ok=True)
with zipfile.ZipFile(ZIP_PATH, "r") as z:
    z.extractall(EXTRACT_DIR)

tifs = list(EXTRACT_DIR.rglob("*.tif")) + list(EXTRACT_DIR.rglob("*.tiff"))
if not tifs:
    raise FileNotFoundError("No GeoTIFF found")
tif_path = tifs[0]
print("Using raster:", tif_path.name)

# ---------------------------------------------------------------------
# 3. Calculate flood severity per district
# ---------------------------------------------------------------------
print("Calculating flood severity for each district...")

with rasterio.open(tif_path) as src:
    bkk = bkk.to_crs(src.crs)

    flood_pct = []
    for idx, row in bkk.iterrows():
        try:
            geom = [mapping(row.geometry)]
            out_image, out_transform = mask(src, geom, crop=True, filled=False)
            data = out_image[0]

            # Count valid pixels and flooded pixels
            # (EOS-RS FPM usually has values > 0 for flooded areas)
            valid = data[~np.isnan(data) & (data > -9999)]
            if len(valid) == 0:
                flood_pct.append(0.0)
                continue
```

```
            # Adjust threshold if needed (most FPMs use 1 or positive values for water)
            flooded = valid[valid > 0]
            pct = (len(flooded) / len(valid)) * 100
            flood_pct.append(pct)
        except Exception:
            flood_pct.append(0.0)

bkk["flood_pct"] = flood_pct

# Classify into severity classes for the legend
def severity_class(pct):
    if pct >= 40:
        return "Very High"
    elif pct >= 20:
        return "High"
    elif pct >= 8:
        return "Moderate"
    elif pct >= 2:
        return "Low"
    else:
        return "Very Low / None"

bkk["severity"] = bkk["flood_pct"].apply(severity_class)

# Colour for each severity class
severity_colors = {
    "Very High":      "#08519c",   # dark blue
    "High":           "#3182bd",
    "Moderate":       "#6baed6",
    "Low":            "#bdd7e7",
    "Very Low / None": "#eff3ff",   # almost white
}

bkk["fill_color"] = bkk["severity"].map(severity_colors)

print("\nDistrict flood severity summary:")
print(bkk["severity"].value_counts())

# ---------------------------------------------------------------------
# 4. Gauge stations (optional but useful)
# ---------------------------------------------------------------------
print("\nFetching gauge stations...")
url_meta = "https://api-v3.thaiwater.net/api/v1/thaiwater30/public/waterlevel_load"
r = requests.get(url_meta, headers={"Accept": "application/json"}, timeout=90)
r.raise_for_status()
rows = r.json()["waterlevel_data"]["data"]

stations = []
for item in rows:
    st = item.get("station") or {}
    lat = st.get("tele_station_lat")
    lon = st.get("tele_station_long")
    if lat is None or lon is None:
        continue
    banks = [b for b in [st.get("min_bank"), st.get("left_bank"), st.get("right_bank")] if b is not None]
```

```
    bank_ref = min(banks) if banks else None
    stations.append({
        "station_id": st.get("id"),
        "latitude": float(lat),
        "longitude": float(lon),
        "bank_ref": float(bank_ref) if bank_ref is not None else None,
    })

df_st = pd.DataFrame(stations)

def fetch_wl(sid):
    url = (f"https://api-v3.thaiwater.net/api/v1/thaiwater30/public/waterlevel_graph"
           f"?station_type=tele_waterlevel&station_id={sid}"
           f"&start_date={TARGET_DATE}&end_date={TARGET_DATE}")
    try:
        resp = requests.get(url, headers={"Accept": "application/json"}, timeout=12)
        if resp.status_code != 200:
            return sid, None
        data = resp.json()
        graph = data.get("data", {}).get("graph_data") or data.get("graph_data") or []
        values = [float(g["value"]) for g in graph if g.get("value") not in (None, "")]
        return sid, values[-1] if values else None
    except Exception:
        return sid, None

results = {}
with ThreadPoolExecutor(max_workers=MAX_WORKERS) as ex:
    futures = {ex.submit(fetch_wl, sid): sid for sid in df_st["station_id"]}
    for fut in as_completed(futures):
        sid, wl = fut.result()
        results[sid] = wl

df_st["waterlevel"] = df_st["station_id"].map(results)

def classify(row):
    wl, bank = row["waterlevel"], row["bank_ref"]
    if pd.isna(wl) or pd.isna(bank):
        return "No data"
    diff = wl - bank
    if diff >= 0.3:
        return "Over-bank"
    elif diff >= -0.3:
        return "Near bank"
    return "Normal"

df_st["status"] = df_st.apply(classify, axis=1)

gdf_st = gpd.GeoDataFrame(
    df_st,
    geometry=[Point(xy) for xy in zip(df_st.longitude, df_st.latitude)],
    crs="EPSG:4326"
)
# Keep stations roughly in Bangkok area
gdf_st = gdf_st.cx[100.3:100.95, 13.5:14.0]
```

```
station_colors = {
    "Normal": "#2ecc71",
    "Near bank": "#e67e22",
    "Over-bank": "#8e44ad",
    "No data": "#bdc3c7",
}
gdf_st["color"] = gdf_st["status"].map(station_colors)

# ---------------------------------------------------------------------
# 5. Plot
# ---------------------------------------------------------------------
fig, ax = plt.subplots(figsize=(13, 15))

with rasterio.open(tif_path) as src:
    # Optional: show original raster faintly underneath
    show(src, ax=ax, cmap="Blues", alpha=0.25, zorder=1)

    bkk_plot = bkk
    st_plot  = gdf_st.to_crs(src.crs) if gdf_st.crs != src.crs else gdf_st

    # Districts FILLED by severity colour
    for severity, color in severity_colors.items():
        subset = bkk_plot[bkk_plot["severity"] == severity]
        if len(subset):
            subset.plot(
                ax=ax,
                facecolor=color,
                edgecolor="black",
                linewidth=0.9,
                alpha=0.85,
                zorder=2
            )

    # Gauge stations on top
    st_plot.plot(
        ax=ax,
        color=st_plot["color"],
        markersize=26,
        edgecolor="black",
        linewidth=0.4,
        alpha=0.95,
        zorder=4
    )

    # District labels
    for _, row in bkk_plot.iterrows():
        c = row.geometry.centroid
        ax.annotate(
            text=row["amp_en"],
            xy=(c.x, c.y),
            ha="center", va="center",
            fontsize=6.2, color="black", weight="medium",
            zorder=5
        )
```

```
    minx, miny, maxx, maxy = bkk_plot.total_bounds
    pad = 0.012
    ax.set_xlim(minx - pad, maxx + pad)
    ax.set_ylim(miny - pad, maxy + pad)

ax.set_title(
    "Bangkok Flood Severity by District\n"
    "26–27 September 2026  |  EOS-RS Satellite + ThaiWater Stations",
    fontsize=13, pad=14
)
ax.set_xlabel("Longitude")
ax.set_ylabel("Latitude")
ax.set_aspect("equal")
ax.grid(True, linestyle="--", alpha=0.25)

# Legend – District severity
legend_districts = [
    Patch(facecolor=severity_colors["Very High"], edgecolor="black", label="Very High (≥40%)"),
    Patch(facecolor=severity_colors["High"],      edgecolor="black", label="High (20–40%)"),
    Patch(facecolor=severity_colors["Moderate"],  edgecolor="black", label="Moderate (8–20%)"),
    Patch(facecolor=severity_colors["Low"],       edgecolor="black", label="Low (2–8%)"),
    Patch(facecolor=severity_colors["Very Low / None"], edgecolor="black", label="Very Low / None (<2%)"),
]

# Legend – Stations
legend_stations = [
    Line2D([0], [0], marker="o", color="w", markerfacecolor="#2ecc71",
           markersize=9, markeredgecolor="black", label="Station – Normal"),
    Line2D([0], [0], marker="o", color="w", markerfacecolor="#e67e22",
           markersize=9, markeredgecolor="black", label="Station – Near bank"),
    Line2D([0], [0], marker="o", color="w", markerfacecolor="#8e44ad",
           markersize=9, markeredgecolor="black", label="Station – Over-bank"),
]

legend1 = ax.legend(handles=legend_districts, loc="lower left",
                    title="District Flood Severity", fontsize=9, title_fontsize=10)
ax.add_artist(legend1)
ax.legend(handles=legend_stations, loc="lower right",
          title="Gauge Stations", fontsize=9, title_fontsize=10)

plt.tight_layout()
plt.savefig(OUTPUT_PNG, dpi=300, bbox_inches="tight")
print(f"\nSaved: {OUTPUT_PNG}")
plt.show()
print("Finished.")
```